LAPORAN PRAKTIKUM

STRUKTUR DATA

STACK

Disusun Oleh :

Nama	: Nabila Khairunnisa
Nim	: 2511531003
Dosen Pengampu	: Dr. Wahyudi, S.T, M.T
Asisten Praktikum	: M. Galid Avero

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Puji Syukur atas kehadirat Allah SWT yang telah melimpahkan Rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum struktur data dengan judul "Stack" dengan baik dan tepat waktu. Dalam menyelesaikan laporan ini saya banyak mendapat arahan dan bimbingan, oleh karena itu saya ingin mengucapkan terimakasih kepada

1. Bapak Dr. Wahyudi, S.T, M.T selaku dosen pengampu
2. Uda M. Galid Avero selaku asisten labor
3. Orang tua yang senantiasa mendoakan
4. Teman teman yang selalu memotivasi

Penulis menyadari bahwa laporan ini jauh dari kata sempurna. Oleh sebab itu, penulis sangat membuka diri apabila ada yang ingin memberikan kritikan dan saran yang sifatnya membantu, penulis akan sangat senang menerima. Tujuannya agar untuk kedepannya bisa menyempurnakan laporan.

Padang, 8 April 2026

Nabila Khairunnisa

DAFTAR ISI

KATA PENGANTAR ii

DAFTAR ISI iii

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Manfaat Praktikum 1

BAB II PEMBAHASAN 2

2.1 Deskripsi Praktikum 2
2.2 Langkah Langkah Praktikum 2

BAB III KESIMPULAN 9

3.1 Kesimpulan 9
3.2 Saran 9

DAFTAR PUSTAKA 10

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan cara untuk menyimpan dan mengorganisir data dalam sebuah program komputer. Tujuannya adalah agar dapat diakses dan diproses dengan lebih efisien. Struktur data yang umum digunakan adalah berbentuk stack. Struktur data stack merupakan struktur data linear yang bekerja berdasarkan prinsip LIFO (Last In First Out), yaitu data yang terakhir dimasukkan ke dalam stack akan menjadi data yang pertama kali diambil atau dikeluarkan dari tumpukan.

Kelebihan dan kekurangan Stack :

- Kelebihan : Struktur data stack sangat sederhana dan efisien untuk skenario LIFO, Memory digunakan dengan efisien, Implementasi stack juga cenderung mudah dan cepat, membuat kode lebih ringkas dan mudah dibaca.
- Kekurangan : ukuran biasanya tetap, dapat terjadi *stack overflow* jika terlalu banyak elemen dimasukkan, stack tidak mendukung akses acak. [1]

Dalam istilah pemrograman, upaya menambahkan elemen pada struktur data stack disebut dengan **push**. Sedangkan proses menghapus atau menghilangkan elemen data dari stack disebut **pop**. Selain itu IsEmpty digunakan untuk memeriksa apakah stack kosong, IsFull untuk memeriksa apakah stack sudah penuh serta peek digunakan untuk mendapatkan nilai elemen teratas tanpa menghapusnya [2]

Dalam struktur data stack ada dua kondisi yang perlu dihindari yaitu underflow (keaadaan dimana kita mencoba mengakses atau menghapus data pada stack yang kosong) dan overflow(keadaan di mana ruang memori yang dialokasikan untuk struktur data stack sudah penuh namun masih dilakukan operasi penyisipan elemen). [3]

1.2 Tujuan Praktikum

1. Mampu memahami konsep dasar stack.
2. Mampu mengimplementasikan operasi operasi dasar stack
3. Melatih kemampuan dalam menyusun program yang dapat mengelola data secara terstruktur.
4. Mengembangkan kemampuan analisis dalam menyelesaikan permasalahan.

1.3 Manfaat Praktikum

1. Mahasiswa mampu memahami cara kerja stack
2. Meningkatkan keterampilan mahasiswa dalam membuat program yang menggunakan stack.
3. Mahasiswa mampu menganalisi dalam menyelesaikan masalah

BAB II

PEMBAHASAN

2.1 Deskripsi Praktikum

Praktikum Struktur data pada pekan ini adalah membuat Stack yang diimplementasikan menggunakan array melalui class StackArray yang menyediakan operasi dasar seperti push, pop, dan peek. Selanjutnya, class **StackArrayDriver** digunakan sebagai program utama untuk menguji fungsi-fungsi stack. Selain itu, juga terdapat 3 class yang membahas penerapan stack dalam berbagai kasus.

2.2 Langkah Langkah Praktikum

1) Buatlah package terlebih dahulu dengan mengklik kanan di folder PrakSD2026_C_2511531003/src , pilih new dan klik package. Setelah itu beri nama pada package tanpa huruf kapital, karakter khusus serta tanpa “space”. lalu “finish”.

2) Klik kanan pada package pekan3_2511531003 yang sudah dibuat sebelumnya, pilih “New”, lalu pilih class. Buat nama “StackArray_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini digunakan untuk mengelola stack berbasis array.

3) Tuliskan program seperti berikut.

```java
package pekan3_2511531003;

public class stackArray_2511531003 {
    static final int MAX_1003 = 1000;
    int top;
    int a[] = new int [MAX_1003];
    boolean isEmpty_1003()
    {
        return (top < 0);
    }
    stackArray_2511531003()
    {
        top = -1;
    }
    boolean push (int x)
    {
        if (top >= (MAX_1003 - 1)) {
            System.out.println ("Stack Overflow");
            return false;
        }
        else {
            a[++top] = x;
            System.out.println (x + " dimasukkan dalam stack");
            return true;
        }
    }
    int pop()
    {
        if (top < 0) {
            System.out.println ("Stack Underflow");
            return 0;
        }
        else {
            int x = a[top--];
            return x;
        }
    }
```

```java
    int peek()
    {
        if (top < 0) {
            System.out.println ("Stack Underflow");
            return 0;
        }
        else {
            int x = a[top];
            return x;

        }
    }
    void print () {
        for (int i = top;i>-1;i--) {
            System.out.print (" " + a[i]);
        }
    }
}
```

Gambar 2.2.1 kode program

- *MAX_1003 = 1000* artinya ukuran maksimal stack (1000 elemen).
- *Top* berfungsi untuk penanda indeks elemen paling atas dalam stack.
- *a[]* artinya array menyimpan elemen stack
- method *isEmpty_1003()* ini untuk mengecek apakah stack kosong. Jika *top < 0*, artinya tidak ada data dalam stack.
- Constructor *stackArray_2511531003()* untuk menginisialisasi stack dalam keadaan kosong sehingga top dimulai dari -1.
- Method *push (int x)* berfungsi untuk memasukkan elemen ke stack.
- Method *pop()* berfungsi untuk mengambil dan menghapus elemen teratas stack.
- Method *peek()* berfungsi untuk melihat elemen paling atas tanpa menghapusnya.
- Method *print()* untuk menampilkan semua isi stack dari elemen paling atas sampai elemen paling bawah.

4) Buat lah class kedua dengan nama "stackArrayDriver_2511531003" dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa "spasi", lalu centang tanda "public static void main (string[] args)" lalu "finish". Class ini digunakan untuk menjalankan operasi pada class *stackArray_2511531003*.

5) Tuliskan program seperti berikut.

```java
1  package pekan3_2511531003;
2
3  public class stackArrayDriver_2511531003 {
4
5      public static void main(String[] args) {
6          stackArray_2511531003 s = new stackArray_2511531003();
7          s.push(10);
8          s.push(20);
9          s.push(30);
10         System.out.println (s.pop() + " dikeluarkan dari stack");
11         System.out.println ("Elemen teratas adalah :" + s.peek());
12         System.out.println ("Elemen pada stack :");
13         s.print();
14     }
15 }
16
```

Gambar 2.2.2 Kode Program stackArrayDriver_2511531003

- Method pada no 6 berfungsi untuk membuat objek stack bernama s dari class *stackArray_2511531003*
- method pada no 7 sampai 9 digunakan untuk menambahkan elemen ke stack (push).

- Method pada no 10 digunakan untuk menghapus elemen teratas(pop). Pop() akan mengeluarkan elemn paling atas yaitu 30.
- Method pada no 11 digunakan untuk melihat elemen teratas tanpa menghapusnya (peek)
- Method no 12 dan 13 berfungsi untuk menampilkan semua elemen stack dari atas ke bawah.

6) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut

```
<terminated> stackArrayDriver_2511531003 [Java A
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah :20
Elemen pada stack :
 20 10
```

Gambar 2.2.3 Output program

7) Class ketiga dengan nama "NilaiMaksimum_2511531003" dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa "spasi", lalu "finish". Class ini berfungsi untuk mencari nilai maksimum yang ada di dalam stack, tanpa menghilangkan isi stack secara permanen.

8) Tuliskan program seperti berikut.

```java
package pekan3_2511531003;
import java.util.Stack;
public class NilaiMaksimum_2511531003 {
public static int max (Stack<Integer> s_1003) {
    Stack <Integer> backup = new Stack <Integer> ();
    int maxValue = s_1003.pop();
    backup.push (maxValue);
    while (!s_1003.isEmpty ()) {
        int next = s_1003.pop ();
        backup.push (next);
        maxValue = Math.max (maxValue, next);
    }
    while (!backup.isEmpty()) {
    s_1003.push (backup.pop());
    }
    return maxValue;
    }
    public static void main(String[] args) {
        Stack <Integer> s_1003 = new Stack <Integer> ();
        s_1003.push (70);
        s_1003.push (12);
        s_1003.push (20);
        System.out.println ("isi stack "+s_1003);
        System.out.println ("Stack teratas "+s_1003.peek());
        System.out.println ("Nilai maksimum "+max(s_1003));
    }
}
```

Gambar 2.2.4 Kode Program

- Method *max(Stack<Integer> s_1003)* digunakan untuk mencari nilai terbesar (maksimum) dalam stack. *Max()* tidak merusak isi stack, karena datanya dipindahkan lalu dikembalikan lagi.
- Method *main()* untuk membuat stack integer *s_1003,* menambahkan elemen menggunakan push, menampilkan isi stack, menampilkan elemen teratas menggunakan peek, menampilkan nilai maksimum menggunakan method max().

9) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut

```
<terminated> NilaiMaksimum_2511531003 [Java
isi stack [70, 12, 20]
Stack teratas 20
Nilai maksimum 70
```

Gambar 2.2.5 Output Program

10) Class ke empat dengan nama "siswaStack_2511531003" dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa "spasi", lalu "finish". Class ini berfungsi sebagai class objek/ data untuk merepresentasikan seorang siswa/mahasiswa.

```java
package pekan3_2511531003;
import java.util.ArrayList;
class Siswa_2511531003 {
      String nama;
      int nim;

      public Siswa_2511531003 (String nama, int nim) {
         this.nama = nama;
         this.nim = nim;
      }
      @Override
         public String toString () {
             return "Nim: " + nim + ", Nama: " + nama;
         }
}
      public class SiswaStack_2511531003 {
          private ArrayList <Siswa_2511531003> stack;

          public SiswaStack_2511531003 () {
              stack = new ArrayList <>();
          }

          public void push (Siswa_2511531003 mhs) {
              stack.add (mhs);
          }
          public Siswa_2511531003 pop () {
              if (!isEmpty()) {
                  return stack.remove(stack.size () - 1);
              }
              return null;
          }
          public Siswa_2511531003 peek() {
              if (!isEmpty()) {
                  return stack.get(stack.size() - 1);
              }
```

```java
36                  return null;
37              }
38              public boolean isEmpty () {
39                  return stack.isEmpty ();
40              }
41              public void tampilkanSiswa () {
42                  for (int i = stack.size () - 1; i >= 0; i--) {
43                      System.out.println (stack.get(i));
44                  }
45                  }
46              public static void main(String[] args) {
47                  SiswaStack_2511531003 studentStack = new SiswaStack_2511531003();
48
49                  Siswa_2511531003 mhs1 = new Siswa_2511531003 ("Ali", 1);
50                  Siswa_2511531003 mhs2 = new Siswa_2511531003 ("boby", 2);
51                  Siswa_2511531003 mhs3 = new Siswa_2511531003 ("Charles", 3);
52
53                  studentStack.push (mhs1);
54                  studentStack.push (mhs2);
55                  studentStack.push (mhs3);
56
57                  System.out.println ("Siswa di dalam stack:");
58                  studentStack.tampilkanSiswa();
59
60                  System.out.println ("Siswa teratas " + studentStack.peek());
61                  System.out.println ("mengeluarkan siswa teratas dari stack: " +studentStack.pop());
62                  System.out.println ("daftar siswa setelah di pop :");
63                  studentStack.tampilkanSiswa();
64              }
65              }
```

Gambar 2.2.6 Kode Program

- *String nama* dan *int nim* untuk menyimpan nama siswa dan Nim siswa.
- Method no 7 sampai 9 adalah bagian constructor yaitu digunakan untuk mengisi nilai nama dan nim saat objek dibuat.
- Method *toString()* digunakan untuk mengubah objek siswa menjadi teks agar saat ditampilkan akan terbaca rapi.
- *Public class SiswaStack* berfungsi untuk implementasi stack untuk menyimpan objek *Siswa_2511531003* menggunakan ArrayList.
- Construktor berfungsi untuk menginisialisasi stack agar siap digunakan.
- Method *push()* berfungsi untuk menambahkan data siswa ke stack.
- Method *pop()* berfungsi untuk menghapus sekaligus mengembalikan siswa yang berada di posisi paling atas stack. Jika stack kosong maka akan mengembalikan null. Jika tidak kosong maka akan menghapus elemen terakhir (top).
- Method *peek()* digunakan utnuk melihat siswa paling atas tanpa menghapusnya.
- Method *isEmpty()* untuk mengecek apakah stack kosong atau tidak.
- Method *tampilkanSiswa()* berfungsi untuk menampilkan semua siswa dalam stack, dimulai dari elemen paling atas hingga paling bawah.
- Method *main()* untuk melakukan testing stack.

11) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut

```
<terminated> SiswaStack_2511531003 [Java Application] C:\Users\lenovo\.p2\pool\plugins
Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: boby
Nim: 1, Nama: Ali
Siswa teratas Nim: 3, Nama: Charles
mengeluarkan siswa teratas dari stack: Nim: 3, Nama: Charles
daftar siswa setelah di pop :
Nim: 2, Nama: boby
Nim: 1, Nama: Ali
```

Gambar 2.2.7 Output Program

12) Class kelima dengan nama "StackPostfix_2511531003" dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa "spasi", lalu "finish". Class ini berfungsi untuk menghitung hasil ekspresi postfix menggunakan strukur data stack. Postfix adalah bentuk penulisan operasi matematika dimana operator ditulis setelah operand.

```
1  package pekan3_2511531003;
2  import java.util.Scanner;
4
5  public class StackPostfix_2511531003 {
6      public static int postfixEvaluate (String expression) {
7          Stack <Integer> s_1003 = new Stack <Integer>();
8          Scanner input = new Scanner (expression);
9          while (input.hasNext()) {
10             if (input.hasNextInt()) {   //an operand (integer)
11                 s_1003.push(input.nextInt());
12             }else {
13                 String operator = input.next();
14                 int operand2_1003 = s_1003.pop();
15                 int operand1_1003 = s_1003.pop();
16                 if (operator.equals("+")) {
17                     s_1003.push(operand1_1003 + operand2_1003);
18                 } else if (operator.equals("-")) {
19                     s_1003.push(operand1_1003 - operand2_1003);
20                 } else if (operator.equals("*")) {
21                     s_1003.push(operand1_1003 * operand2_1003);
22                 } else {
23                     s_1003.push(operand1_1003 / operand2_1003);
24             }
25         }
26     }
27         input.close();
28         return s_1003.pop ();
29 }
30     public static void main (String [] args) {
31         System.out.println ("hasil postfix= "+postfixEvaluate ("5 2 4 * + 7 -"));
32     }
33 }
34
```

Gambar 2.2.8 Kode Program

- Method *postfixEvaluate(String expression)* berfungsi untuk menghitung nilai ekspresi postfix.
- *Stack<Integer>* digunakan untuk menyimpan operand (angka)
- *Scanner* digunakan untuk membaca token dari string ekspresi (angka dan operator).
- Loop berjalan selama masih ada token (angka/operator) yang bisa dibaca.
- Jika token berupa integer, maka angka tersebut dimasukkan ke dalam stack.
- Jika token bukan angka, maka dianggap sebagai operator.
- Method no 16 sampai 23 adalah proses operasi matematika. Kemudian hasil operasi dimasukkan kembali ke stack.

- *Input.close* dan *return s_1003.pop* yaitu setelah semua token di proses, stack hanya tersisa 1 nilai yaitu hasil akhir postfix.
- Method *main()* untuk menjalankan program dan menguji method postfixEvaluate.

13) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut

```
<terminated> StackPostfix_2511531003 [Java
hasil postfix= 6
```

Gambar 2.2.9 Output Program

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa stack merupakan struktur data yang bekerja dengan prinsip LIFO (Last In First Out), yaitu data yang terakhir dimasukkan akan menjadi data yang pertama dikeluarkan. Pada praktikum ini, terdapat beberapa operasi dasar stack seperti push untuk menambahkan data, pop untuk menghapus data teratas, peek untuk melihat elemen paling atas tanpa menghapusnya, dan top untuk untuk penanda indeks elemen paling atas dalam stack.

3.2 Saran

Untuk menambah pengembangan praktikum di masa mendatang disarankan untuk memperluas dan mengembangkan program dengan penerapan stack pada kasus nyata.

DAFTAR PUSTAKA

[1] N. Shiba, "Struktur Data Stack: Pengertian, Jenis dan Contoh," IDS DIGITAL COLLEGE STMIK INDO DAYA SUVANA, 13 Maret 2023. [Online]. Available: https://ids.ac.id/struktur-data-stack-pengertian-jenis-dan-contoh/. [Accessed 12 April 2026].

[2] Prasatya, "Apa Itu Stack dalam Ilmu Pemrograman," CODEPOLITAN, 4 Juni 2025. [Online]. Available: https://www.codepolitan.com/blog/apa-itu-stack-dalam-ilmu-pemrograman/https://www.codepolitan.com/blog/apa-itu-stack-dalam-ilmu-pemrograman/. [Accessed 12 April 2026].

[3] Trivusi, "Struktur Data Stack: Pengertian, Karakteristik, dan Kegunaannya," Trivusi, 16 September 2022. [Online]. Available: https://www.trivusi.web.id/2022/07/struktur-data-stack.html#google_vignette. [Accessed 12 April 2026].